

COMSATS University Islamabad, Lahore Campus

Department of Computer Science

CSC241 – Object Oriented Programming (Java)

Theory Assignment 1: Object Oriented Analysis of a Smart Campus Allocation System

Field	Detail
Registration No. / Section	SP26-BAI-045 / BAI
Instructor	Dr Muhammad Shahid Bhatti
Assessment / Due date	Theory Assignment 1 (CLO1), 10 marks — Due 26 September 2026
Student-specific condition	Last digit 5 → Emergency continuity: a building closure may trigger cancellation, relocation, and notifications across many reservations

Task 1 — Stakeholder and Requirement Analysis

1.1 Stakeholders and Needs

Stakeholder	Primary need from the system
Student	Fair and reasonably timely access to book a space for individual study or group work.
Faculty member	Guaranteed priority when booking a space for scheduled teaching or an urgent academic activity.
Administrator	Visibility into room utilization and the ability to configure approval and priority rules.
Security staff	An accurate, traceable record of who accessed which space and when, for safety and investigation.
Maintenance staff	The ability to immediately block an unsafe room, overriding any existing bookings on it.

1.2 Genuine Requirement Conflicts

- Fair access (students) vs. teaching priority (faculty).** Students expect equal opportunity to book rooms, but faculty need reliable priority for scheduled classes and urgent academic work. Decision needed before design: how much priority faculty receive and whether students keep a protected quota of slots.
- High room utilization (administration) vs. immediate safety blocking (maintenance).** The administration wants rooms booked as often as possible, while maintenance staff must be free to take an unsafe room offline instantly, cancelling or relocating any current booking. Decision needed: safety blocking must be allowed to override utilization targets without waiting for administrator approval. This same conflict re-appears, at a larger scale, under the assigned Emergency continuity condition: a building closure forces many reservations to be cancelled or relocated at once, further reducing utilization but protecting safety and continuity.
- Traceable access records (security) vs. personal privacy (users).** Security staff need to know who used a space and when, but users expect their personal details and the purpose of their visit to stay private from other ordinary users. Decision needed: what a log entry contains and which roles are allowed to view it.

1.3 Facts vs. Assumptions

Facts stated in the scenario:

- Four request types exist (individual, group, class, event), each with different capacity, equipment, and approval rules.
- Six stakeholder categories are named: students, faculty, administrators, security, maintenance, and disability support staff.
- A person may hold more than one role at once, and roles/permissions may change during the semester.

- Emergency events may cancel or relocate confirmed reservations; the system must keep enough information for an audit and must notify affected users.
- The scenario explicitly leaves fair access, recurring-reservation limits, and simultaneous-request handling undefined.

Assumptions introduced for this design:

- A scheduled faculty class request outranks an ordinary student request, but a fixed weekly quota of slots per room is reserved for student use so fairness is preserved.
- Simultaneous requests for the same space are resolved by request timestamp, with role priority as a tie-break (see Task 5).
- Recurring reservations are capped at a configurable maximum number of consecutive weeks, so one user cannot permanently occupy a room.
- Access-log entries are visible only to security staff and administrators, never to ordinary users.
- Every cancellation or relocation caused by an emergency event is written to the audit log before the affected reservation is closed, and every affected user is notified automatically.

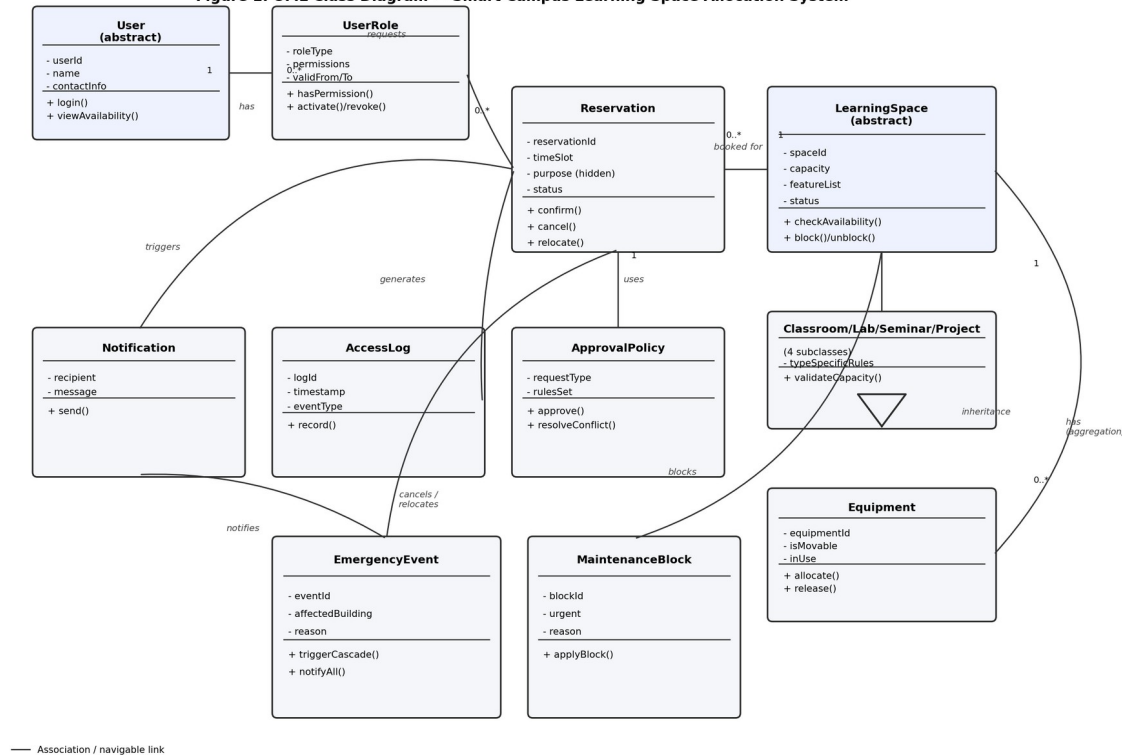
Task 2 — Object Oriented Conceptual Model

Nine classes are proposed (one of which — LearningSpace — expands into four concrete subclasses through inheritance, giving ten concrete building blocks in total).

Class	Responsibility	Key attributes / operations
User (abstract)	Represents any person who interacts with the system.	userId, name, contactInfo; login(), viewAvailability()
UserRole	Encapsulates one role and its permissions, held by a User; a user may hold several at once.	roleType, permissions; hasPermission(), activate()/revoke()
LearningSpace (abstract)	Represents a bookable physical space and its current status.	spaceId, capacity, featureList, status; checkAvailability(), block()/unblock()
Classroom / Laboratory / Seminar Room / Project Space	Four subclasses of LearningSpace, each enforcing its own capacity/equipment rules.	typeSpecificRules; validateCapacity()
Equipment	Represents one movable or fixed item that a space may need.	equipmentId, isMovable; allocate(), release()
Reservation	Represents one booking request through its full life cycle.	reservationId, timeSlot, purpose (hidden), status; confirm(), cancel(), relocate()
ApprovalPolicy	Encapsulates the approval rule for one request type, so rules can change independently of Reservation.	requestType, rulesSet; approve(), resolveConflict()
AccessLog	Records one auditable event (entry, cancellation, relocation) for traceability.	logId, timestamp, eventType; record()
Notification	Represents one message sent to an affected user.	recipient, message; send()
EmergencyEvent	Coordinates a building closure: identifies affected reservations and triggers their cancellation/relocation and notification.	eventId, affectedBuilding, reason; triggerCascade(), notifyAll()
MaintenanceBlock	Represents an urgent or planned block placed on a space by maintenance staff.	blockId, urgent, reason; applyBlock()

2.1 UML Class Diagram

Figure 1: UML Class Diagram — Smart Campus Learning Space Allocation System



Multiplicities: a User holds 0..* UserRole objects; a UserRole (acting on behalf of the user) may make 0..* Reservations; each Reservation books exactly 1 LearningSpace; a LearningSpace aggregates 0..* Equipment items; a Reservation uses exactly 1 ApprovalPolicy and generates 0..* AccessLog entries and 0..* Notifications; an EmergencyEvent can cancel/relocate 0..* Reservations and issues 0..* Notifications; a MaintenanceBlock applies to 1 LearningSpace. LearningSpace generalizes into Classroom, Laboratory, Seminar Room, and Project Space (inheritance), which is justified because each subclass enforces different capacity, equipment, and approval rules while sharing the same booking behaviour.

2.2 Separation of Responsibilities

No single class controls the whole system. Reservation only manages its own life cycle and never decides approval rules directly — that logic lives in ApprovalPolicy, so a new request type can be added without changing Reservation. LearningSpace only manages its own availability and blocking, not who has booked it. EmergencyEvent coordinates a building-wide cascade, but it delegates the actual state change to each Reservation's own cancel()/relocate() operations, delegates communication to Notification, and delegates the audit trail to AccessLog, rather than performing all of that work itself. This keeps each class responsible for one concern and keeps the design changeable as rules evolve.

Task 3 — Abstraction, Encapsulation and Invariants

3.1 What Three Key Classes Abstract

- **Reservation** — abstracts the real-world act of booking a space for a period of time, reducing it to a status, a time slot, and a purpose, while ignoring irrelevant real-world detail such as how the user physically travelled to the room.
- **LearningSpace** — abstracts a physical room down to the properties that matter for allocation: its capacity, its features, and whether it is currently available, hiding physical details such as furniture layout.
- **EmergencyEvent** — abstracts a building closure into a single trigger that produces a consistent cascade of cancellations, relocations, and notifications, hiding the internal complexity of coordinating many reservations at once.

3.2 Hidden Information and Controlled Operations

Class	Information hidden	How it may change
Reservation	Exact status field and the requester's purpose/identity, which are not exposed to other ordinary users.	Only through confirm(), cancel(), and relocate(); no external class may set status directly.
LearningSpace	Internal safety/maintenance detail; other classes see only an availability flag.	Only through block() and unblock(), callable only by a MaintenanceBlock or EmergencyEvent acting through the maintenance/administrator role.
EmergencyEvent	The internal logic that determines which reservations are affected by a closure.	Only through triggerCascade(), which atomically updates affected reservations and queues notifications.

3.3 Invariants

1. A LearningSpace may not have two Confirmed reservations with overlapping time slots.
2. A Reservation's status may only move along the defined life cycle (Requested → Approved/Rejected → Confirmed → Completed/Cancelled/Relocated); no state may be skipped.
3. If a LearningSpace has an active MaintenanceBlock, no new Reservation for it may be confirmed until the block is lifted.
4. Every Cancelled or Relocated reservation must have a matching AccessLog entry recorded before it is treated as closed, so an audit trail always exists — this directly supports the Emergency continuity condition.
5. When an EmergencyEvent closes a building, no Reservation may be left pointing to a closed space in the Confirmed state; each one must be moved to Cancelled or Relocated as part of the same operation.

Because status-changing fields are private and can only be modified through the methods listed above, no other class can push an object into a state that violates these rules. Encapsulation therefore concentrates the responsibility for correctness inside the object that owns the data, instead of trusting every caller to update it consistently.

Task 4 — Alternative Designs and Decision

4.1 Two Ways to Represent Multi-Role Users

Option A — Inheritance: User is a base class with subclasses Student, Faculty, Administrator, SecurityStaff, MaintenanceStaff, and DisabilitySupportStaff. A person's type is fixed by which subclass their object was created from.

Option B — Composition / role objects: User is a single concrete class that holds a collection of UserRole objects (e.g., StudentRole, FacultyRole). Permissions are checked by asking the held role objects, and roles can be added or removed at run time.

4.2 Comparison

Criterion	Inheritance	Composition / role objects
Handling multiple simultaneous roles	Poor — a person cannot easily be both Student and TeachingAssistant without multiple inheritance or a duplicate object.	Natural — a User simply holds more than one UserRole at the same time.
Ease of changing permissions at run time	Difficult — changing a person's role usually means replacing their object or its type.	Easy — a UserRole is added or revoked without touching the User object.
Duplicate behaviour / extensibility	New role → new subclass; shared behaviour must be pulled up carefully to avoid duplication.	New role → new UserRole implementation; shared permission-checking logic stays in one place.

4.3 Decision

Composition using role objects is chosen for this scenario. The requirements explicitly state that a person may hold more than one role and that roles and permissions may change during the semester; the composition approach satisfies both directly, while the inheritance approach would require awkward workarounds such as multiple inheritance or maintaining duplicate accounts for the same person.

Task 5 — Behaviour and Edge Cases

5.1 Reservation Life Cycle — State Transition Table

Current state	Trigger	Next state
Requested	ApprovalPolicy approves the request	Approved
Requested	ApprovalPolicy rejects the request	Rejected
Approved	User/system confirms within the allowed window	Confirmed
Confirmed	Time slot passes without incident	Completed
Confirmed	User or administrator cancels	Cancelled
Confirmed	MaintenanceBlock is applied to the space	Cancelled or Relocated
Confirmed	EmergencyEvent closes the building	Cancelled or Relocated
Relocated	New space/time slot is confirmed	Confirmed (for the new space)

5.2 Edge Cases

- **Two simultaneous requests for the same space** — Reservation itself checks the overlap invariant before allowing a confirm(); ApprovalPolicy resolves the tie using timestamp order with role priority as a tie-break, so one request is confirmed and the other is rejected or waitlisted. Enforced by: **Reservation and ApprovalPolicy**.
- **A maintenance block after confirmation** — Maintenance staff create a MaintenanceBlock, which immediately marks the LearningSpace unavailable; the space then instructs every Confirmed reservation on it to cancel() or relocate(), and Notification informs the affected users. Enforced by: **MaintenanceBlock together with LearningSpace and Reservation**.
- **An emergency cancellation or relocation** — An EmergencyEvent is raised for a building; it identifies every Reservation tied to a space in that building, calls cancel() or relocate() on each one, writes an AccessLog entry for every change, and sends a Notification to every affected user, satisfying the Emergency continuity condition. Enforced by: **EmergencyEvent, which orchestrates Reservation, AccessLog, and Notification**.

Task 6 — Independent Exploration

The Strategy Pattern

The Strategy pattern defines a family of interchangeable algorithms behind one common interface, so the class using them does not need to know which specific variant is running. Instead of writing one method full of if/else branches for every case, the class holds a reference to a strategy object and simply calls it; a different strategy can be swapped in without changing the calling class at all. This is exactly the problem that Task 1's four request types create: individual, group, class, and event bookings each need different approval rules, and hard-coding all four inside Reservation would make the class harder to extend every time a new request type appears. Applying the Strategy pattern, ApprovalPolicy becomes an interface with one approve() method, implemented by four concrete strategies —

IndividualApprovalStrategy, GroupApprovalStrategy, ClassApprovalStrategy, and EventApprovalStrategy. Reservation simply calls its assigned strategy's approve() method. A new request type, or a change to one type's rule, can then be added by writing one new strategy class, without editing Reservation or any existing strategy, which directly improves the extensibility criterion discussed in Task 4.

(≈195 words)

References

1. Association for Computing Machinery and IEEE Computer Society, Computing Curricula 2020: Paradigms for Global Computing Education, 2020. <https://www.acm.org/education/curricula-recommendations>
2. Seoul Accord, Section D — Graduate Attributes, especially Section D.4.1 Range of Problem Solving. <https://www.seoulaccord.org/document.php?id=79>
3. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994 (Strategy pattern, Chapter 5).
4. Refactoring.Guru, "Strategy Design Pattern," <https://refactoring.guru/design-patterns/strategy> (accessed September 2026).